

# Citizen Fraud Shield AI

A multi-agency platform for detecting and reporting digital-arrest, UPI, and phishing scams in India

---

## Detailed Project Document

|            |                                                                                                               |
|------------|---------------------------------------------------------------------------------------------------------------|
| Category   | Digital Governance / Cybersecurity / Citizen Safety                                                           |
| Repository | <a href="https://github.com/soham53crtl/citizen-fraud-shield">github.com/soham53crtl/citizen-fraud-shield</a> |
| Backend    | Node.js, Express, MongoDB Atlas, Socket.IO                                                                    |
| Frontend   | React, Vite                                                                                                   |
| Status     | Functional local prototype, automated test suite passing (16/16)                                              |

# 1. Problem Statement

India has seen a sharp rise in “digital arrest” scams, where fraudsters impersonate agencies such as the CBI, Enforcement Directorate, or local police over video call, falsely claim a warrant or FIR exists, and pressure victims into transferring money under threat of arrest. These scams sit alongside longer-standing UPI/OTP phishing, fake courier/customs seizure calls, lottery scams, and job-lure fraud — all of which rely on urgency, secrecy, and impersonation of authority to bypass a victim’s normal judgement.

Citizens currently have no single place to (a) check whether a message or call they received matches known scam patterns before acting, (b) file a structured report that reaches the right investigating authority, and (c) see how their case connects to a wider fraud network. Cyber cells and banks, in turn, lack a shared, real-time view of emerging scam patterns across regions.

# 2. Solution Overview

Citizen Fraud Shield AI is a role-based web platform connecting five stakeholder types — citizens, police officers, cyber analysts, bank officers, and administrators — around a shared fraud-intelligence layer. It combines an AI-assisted scam risk engine, a citizen reporting portal, a fraud-network graph, a geospatial crime-pattern map, and a counterfeit-currency checklist tool into one system.

**Scam Detector** — Citizens paste a suspicious SMS/call transcript/email and get an instant risk score, band (LOW/GUARDED/ELEVATED/CRITICAL), category, and recommended next steps, including the correct helpline (1930) and portal (cybercrime.gov.in).

**Citizen Portal** — Structured multi-channel reporting (SMS, WhatsApp, phone call, email, URL, other) that automatically runs every submission through the risk engine and routes high-risk reports to investigators in real time via Socket.IO.

**Fraud Network Graph** — Visualises links between reported fraud entities (numbers, UPI IDs, accounts) and complaints, helping investigators spot rings rather than isolated incidents.

**Geospatial Crime Map** — Plots complaint locations across Indian states/cities to reveal regional hotspots and emerging patterns over time.

**Counterfeit Currency Check** — A guided checklist for citizens to visually verify banknote security features against known genuine-note characteristics.

# 3. System Architecture

The system follows a conventional three-tier architecture, chosen for reliability and speed of iteration over unnecessary complexity for a prototype of this scope.

| Layer     | Technology                                    | Responsibility                                                        |
|-----------|-----------------------------------------------|-----------------------------------------------------------------------|
| Client    | React + Vite (code-split by route/tab)        | Role-based dashboards, scam detector UI, live map/graph views         |
| API       | Node.js + Express, JWT auth, Zod validation   | Auth, report intake, risk-engine invocation, role authorization       |
| Real-time | Socket.IO (Redis adapter if REDIS_URL is set) | Pushes CRITICAL/ELEVATED fraud alerts live to investigator dashboards |

|                  |                                                                   |                                                                           |
|------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------|
| ML / Risk Engine | Regex signal library + trained Naive Bayes Classifier (backed by) | Classifies (backed by) blends rule-based and statistical signals          |
| Data             | MongoDB Atlas (Mongoose ODM)                                      | Users, Complaints, FraudEntity, Connection, Evidence, Notification, RiskR |
| CI/CD            | GitHub Actions                                                    | Runs the automated test suite and a production build on every push        |

## Citizen Fraud Shield AI — System Architecture

Three-tier architecture: React client, Express API with a blended AI risk engine, MongoDB Atlas, and real-time alerts

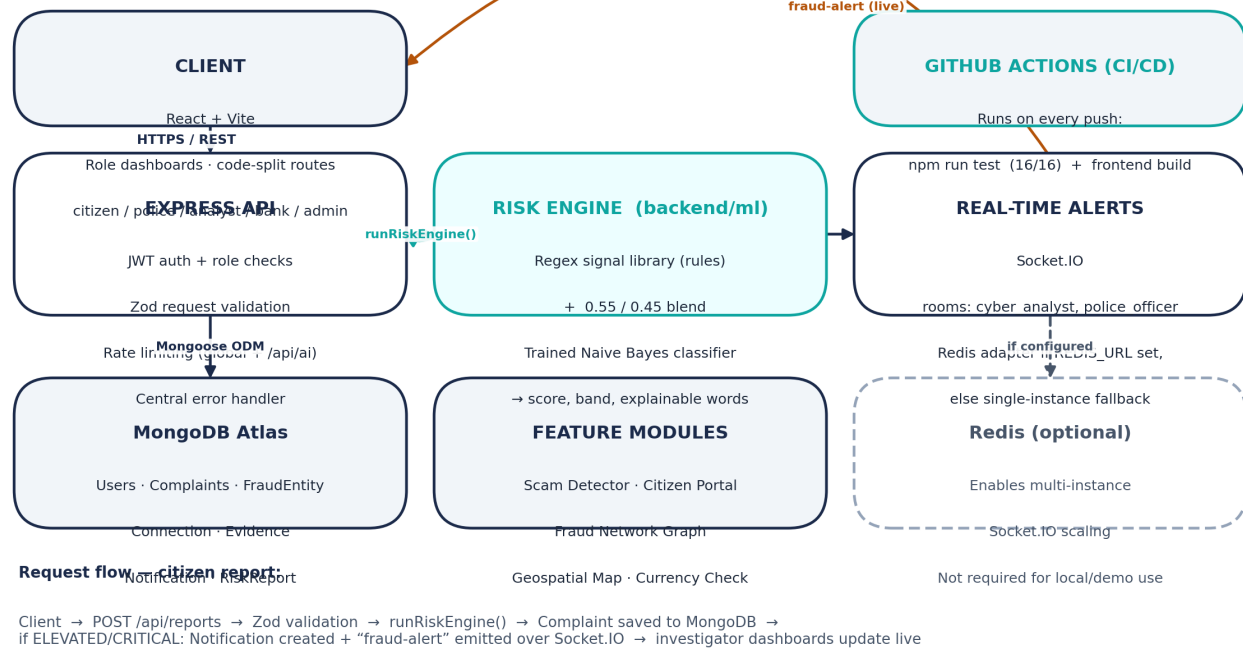


Figure 1. High-level architecture — client, API, blended risk engine, database, and real-time alerting, with the optional Redis scaling path shown dashed.

Request flow for a citizen report: Client → **POST /api/reports** → Zod validation → **runRiskEngine()** (regex + Naive Bayes blend) → Complaint persisted to MongoDB → if band is ELEVATED/CRITICAL, a Notification is created and a **fraud-alert** event is emitted over Socket.IO to the *cyber\_analyst* and *police\_officer* rooms → investigator dashboards update live without a page refresh.

## 4. AI / ML Approach — Risk Engine

The risk engine blends two independent, complementary signal sources rather than relying on either alone:

**1. Rule-based signal library** — a set of interpretable, hand-authored regular expressions for known scam patterns (e.g. “digital arrest”, impersonation of CBI/ED/TRAI, demands for OTP/CVV/UPI PIN, urgency + secrecy framing). These are precise and legally grounded (e.g. explicitly noting that digital arrest has no basis in Indian law) but brittle — they only catch phrasing they were written for.

**2. Trained Multinomial Naive Bayes text classifier** — a genuine statistical model (word-frequency likelihoods with Laplace smoothing) trained on a labelled dataset of scam vs. legitimate message examples. It generalises to phrasing the regex rules never saw, and returns a probability distribution plus the specific words that drove its decision, so every verdict is explainable rather than a black box.

The two scores are combined as **final\_score = 0.55 × regex\_score + 0.45 × classifier\_scam\_probability**, clamped to a 4–98 range and mapped to four bands (LOW / GUARDED / ELEVATED / CRITICAL). Regex

signals are weighted slightly higher because they are deterministic and tied to concrete legal facts, while the classifier's probabilistic output adds coverage for messages that don't match any known pattern.

## Honest limitations of the current model

- The classifier is trained on a small, hand-written example set, not a large corpus of real reported scam messages — it has not yet been evaluated on a held-out test set, so no measured precision/recall/false-positive-rate figure is claimed. We treat this as an open item rather than overstating accuracy.
- The counterfeit-currency module is a guided checklist based on known RBI security features, not a trained computer-vision model — it does not perform microprint or UV-feature image analysis, and is presented to users as such.
- The system currently runs against seeded demo data locally; it has not been validated by an actual police unit, bank, or cyber-cell partner.

## 5. Security, Reliability & Scalability

**Authentication & authorization.** JWT-based auth with role checks (*requireAuth*, *requireRole*) enforced on every sensitive route; citizens can only view their own reports, investigators can view/update case status.

**Input validation.** Zod schemas validate signup, login, scam-analysis, and report-submission payloads at the API boundary, rejecting malformed requests with field-level error messages before they reach business logic.

**Rate limiting.** A global limiter (300 req / 15 min) protects the whole API; the AI analysis endpoint carries an additional, tighter limit (20 req / min) since it is the most compute-sensitive and most likely to be probed or abused.

**Fault isolation.** A React error boundary wraps each dashboard tab so a failure in one module (e.g. the map) cannot blank out the entire application for the user.

**Scalability path.** Socket.IO is wired to use a Redis adapter automatically when a *REDIS\_URL* is configured, enabling real-time alerts to work correctly across multiple backend instances; it falls back cleanly to single-instance mode when Redis is not present, so the same codebase works for a local demo and a horizontally scaled deployment.

**Performance.** Frontend bundle is code-split by route/tab (map, graph, and analytics only load when opened), reducing the initial JS payload from roughly 866 KB to a 236 KB main chunk.

## 6. Testing & CI/CD

The backend has an automated test suite (Node's built-in test runner, no extra framework dependency) covering three areas, all passing at the time of writing:

| Suite                          | What it verifies                                                              | Tests                  |
|--------------------------------|-------------------------------------------------------------------------------|------------------------|
| Naive Bayes classifier         | Correct classification on unseen scam/legit phrasing, full probability output | 5 empty-input handling |
| Risk engine (regex + ML blend) | Correct band assignment, score bounds (4–98), model metadata included         | 4 in every response    |

|                    |                                                                                       |
|--------------------|---------------------------------------------------------------------------------------|
| Validation schemas | Signup/login/analyze/report payloads correctly accepted or rejected with field errors |
|--------------------|---------------------------------------------------------------------------------------|

**Total: 16 / 16 tests passing.** A GitHub Actions workflow ([.github/workflows/ci.yml](#)) runs this full suite plus a production frontend build on every push, so regressions are caught automatically rather than relying on manual checks.

## 7. Role-Based Access

| Role           | Primary capability                                                     |
|----------------|------------------------------------------------------------------------|
| Citizen        | Run the Scam Detector, file structured reports, view own case status   |
| Police Officer | View/triage reports, update case status, view fraud network graph      |
| Cyber Analyst  | Receive real-time CRITICAL/ELEVATED alerts, investigate fraud entities |
| Bank Officer   | View flagged financial-fraud reports relevant to banking channels      |
| Admin          | Full visibility across users, reports, and system analytics            |

## 8. Social Impact

Digital arrest and related impersonation scams disproportionately target people less familiar with how Indian law-enforcement actually operates — particularly older citizens. By giving anyone a free, instant, explainable second opinion on a suspicious message before they act, and by giving investigators a shared, real-time view of emerging patterns rather than isolated complaints, the platform aims to shorten the gap between a scam attempt and a citizen recognising it — the single biggest factor in whether money is lost.

## 9. Roadmap

- Collect real, anonymised scam-message examples to expand the training set and run a proper train/test split to publish a measured precision, recall, and false-positive rate.
- Explore a lightweight, genuinely trained computer-vision model for the counterfeit-currency module (microprint and UV-feature analysis) rather than a manual checklist.
- Deploy a live, publicly reachable instance so the platform can be evaluated hands-on rather than only through screenshots or a local demo.
- Pilot with an actual cyber-cell or bank fraud team to validate report routing and triage against real operational workflows.

## 10. How to Run Locally

- **Backend:** `cd backend && npm install && npm run seed && npm run dev` — starts the API on port 5000.
- **Frontend:** `cd frontend && npm install && npm run dev` — starts the Vite dev server (default port 5173).
- **Tests:** `cd backend && npm run test` — runs the full automated suite.

### Demo credentials (email / password)

| Role           | Email            | Password   |
|----------------|------------------|------------|
| Citizen        | citizen@demo.com | citizen123 |
| Police Officer | officer@demo.com | officer123 |
| Cyber Analyst  | analyst@demo.com | analyst123 |
| Bank Officer   | bank@demo.com    | bank123    |
| Admin          | admin@demo.com   | admin123   |